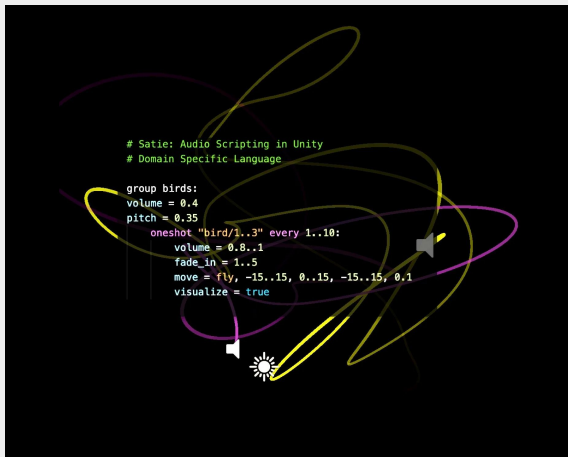


SATIE

<https://github.com/mateolarreaferro/SatieLang>



audio-based **creativity-support tool** for sequencing, generation, positioning, and processing

By: Mateo Larrea

imagine you're a sound designer creating the **ambience of a forest full of birds** (for a game or a meditation app)

you're given this list of requirements for your implementation:

1. Randomly select and play bird sounds from a folder with 5 samples
2. Trigger bird sounds at randomized intervals between 2 and 10 seconds
3. Randomize pitch and volume per event (to avoid fatigue)
4. Move each bird **smoothly** in 3D space (you can use perlin noise or brownian motion)
5. Visualize the trajectory of each moving bird
6. Fade In the entire soundscape in 5 seconds so that it feels gradual
7. Stop all playback and clears visuals after 1 minute

maybe with a digital audio workstation (DAW)?

I asked [Caio Jiacomini](#) - a professional sound designer who has shipped games like Rocket League and Sky:

“Doing this in a DAW is possible but disproportionately tedious: you’re fighting the tool. Random 2–10 s scheduling wants a script-level scheduler, but DAWs push you into clip follow-actions, LFOs, or clunky MIDI tricks with grid quantization and timing jitter. True per-event randomness (independent pitch/volume per bird) is awkward without custom devices (Max for Live, Logic Scripter, JSFX), and smooth 3-axis motion needs stacked LFOs or hacked sample-and-hold with heavy smoothing - there’s no native Perlin/Brownian source. Proper 3D spatialization often requires extra plugins and messy routing, and there’s no concept of “visual trails,” so you’d have to pipe OSC to an external app. Then you still need a global 5 s fade-in, a guaranteed hard stop at 60 s (fighting tails and returns), seeded randomness for reproducible renders, and a sprawling multi-track setup that’s brittle to scale or reuse. In code, this is a few clean loops and parameters; in a DAW, it’s a pile of devices, automation, and workarounds”

maybe with a game engine like [Unity](#)?

- That “simple” program was written in ~280 lines of C# code
- Required extensive knowledge of the language and object oriented programming concepts
- Took longer to write the code than to work on the actual sound design and composition

```
1
2 using System.Collections;
3 using System.Collections.Generic;
4 using UnityEngine;
5
6 public class ForestAmbienceManager : MonoBehaviour
7 {
8     [Header("Timing")]
9     [Tooltip("Min & max time between bird triggers (seconds).")]
10    public Vector2 spawnIntervalRange = new Vector2(2f, 10f);
11
12    [Tooltip("Stop all playback & clear visuals after this many seconds.")]
13    public float stopAfterSeconds = 60f;
14
15    [Header("Global Fade")]
16    [Tooltip("Fade-in duration for the whole soundscape (seconds).")]
17    public float globalFadeInSeconds = 5f;
18
19    [Header("Audio")]
20    [Tooltip("3D audio settings.")]
21    public float minDistance = 2f;
22    public float maxDistance = 20f;
23    public AnimationCurve rolloffCurve = AnimationCurve.EaseInOut(0, 1, 1, 0);
24
25    [Tooltip("Random volume per event (multiplied by global fade).")]
26    public Vector2 perEventVolumeRange = new Vector2(0.35f, 0.85f);
27
28    [Tooltip("Random pitch per event.")]
29    public Vector2 perEventPitchRange = new Vector2(0.85f, 1.15f);
30
31    [Header("Bird Movement")]
32    [Tooltip("Movement radius around origin for birds.")]
33    public float movementRadius = 10f;
34
35    [Tooltip("How fast birds drift (Perlin time scale).")]
36    public float noiseTimeScale = 0.15f;
37
38    [Tooltip("How far a single bird wanders (position scale).")]
39    public float noisePosScale = 8f;
40
41    [Header("Trails")]
42    [Tooltip("Seconds that trails remain visible behind each bird.")]
43    public float trailTime = 3.5f;
44
45    [Tooltip("Start and end width of trails.")]
46    public Vector2 trailWidth = new Vector2(0.15f, 0.0f);
```

See full code [here](#)

maybe with audio middleware like Wwise?

- **Heavy Integration:** Authoring app, SDK, Unity plugin, WAAPI, SoundBank paths.
- **Fragile C# ↔ Wwise Wiring:** PostEvent/RTPC/SetPosition/callbacks are easy to mis-connect.
- **Messy Version Control**
- **Two-App Debugging:** Unity + Wwise Profiler slows iteration.
- **No Architecture for Movement:** emitters don't move themselves; requires per-frame motion logic.
- **Runtime pitfalls:** voice limits, attenuation quirks, and plugin/licensing hurdles for advanced spatialization.
- **Steep learning curve + more complex CI/CD** (headless bank gen).
- **Team Friction:** coordinating versions and banks - often more hassle than a small, deterministic C# system.

maybe with a CST like Satie?

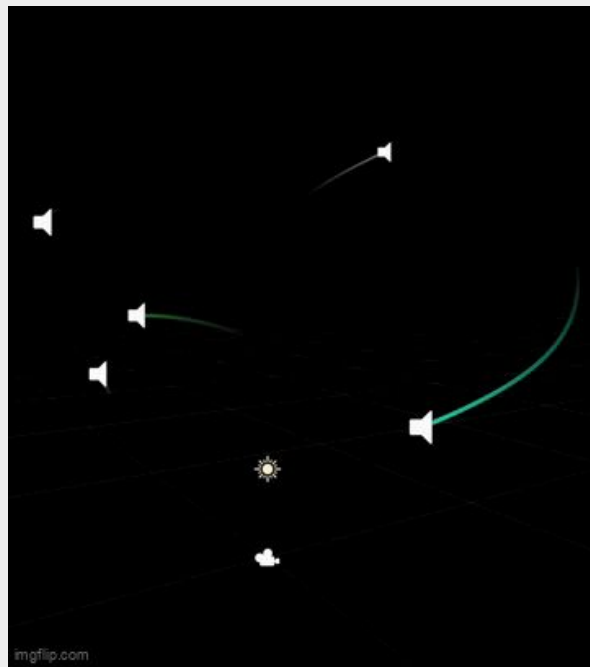
Satie Code

```
comment
Randomly selects and plays bird sounds from a folder with 5 samples
Triggers birds at randomized intervals between 2 and 10 seconds.
Applies randomized pitch and volume per event.
Moves each bird smoothly in 3D space (you can use perlin noise or brownian motion)
Draws a visual trail for each moving bird.
Fade In the entire soundscape in 5 seconds
Stops all playback and clears visuals after 60 seconds.
endcomment

5 * oneshot "bird/1to5" every 2to10:
  fade_in = 5
  volume = 0.1to0.5
  pitch = 0.5to1.1
  move = fly, -10to10, 0to20, -10to10, 0.5to1
  visual = trail
  duration = 60
```

7 lines of code that do what C# does in almost 300. See .sat script [here](#)

Generated Scene



what is **Satie**?

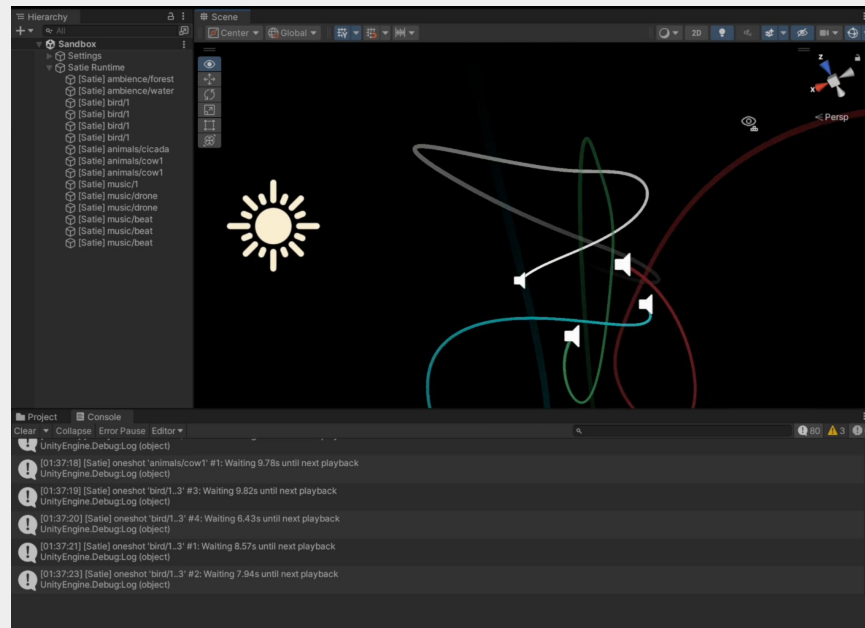
Satie is a creativity-support tool and lightweight language for rapid prototyping of dynamic audio environments. It gives musicians and sound designers with limited coding experience a clear paradigm to build soundscapes, spatial music, video game audio, and [holodeck](#)-style interactions using simple, composable behaviors.

Key Principles of Satie

- Simple, readable syntax
- Leans toward natural language
- Sound Designer-friendly terminology (familiar to musicians)
- Easy, lightweight installation
- **Low floor, high ceiling** (welcoming for beginners, powerful for experts)

How to use Satie? → through code

- **Code-first, end-to-end:** The user script every stage: authoring, auditioning, debugging, and export using the language
- **Live editing in runtime:** Ability to modify the script **while play mode is running**; changes apply instantly (no stop/recompile/relaunch cycle).
- **Tight feedback loop:** The user hears and sees results immediately, fast iteration
- **Same script, multiple contexts:** The code the user writes for prototyping is the code they can ship.



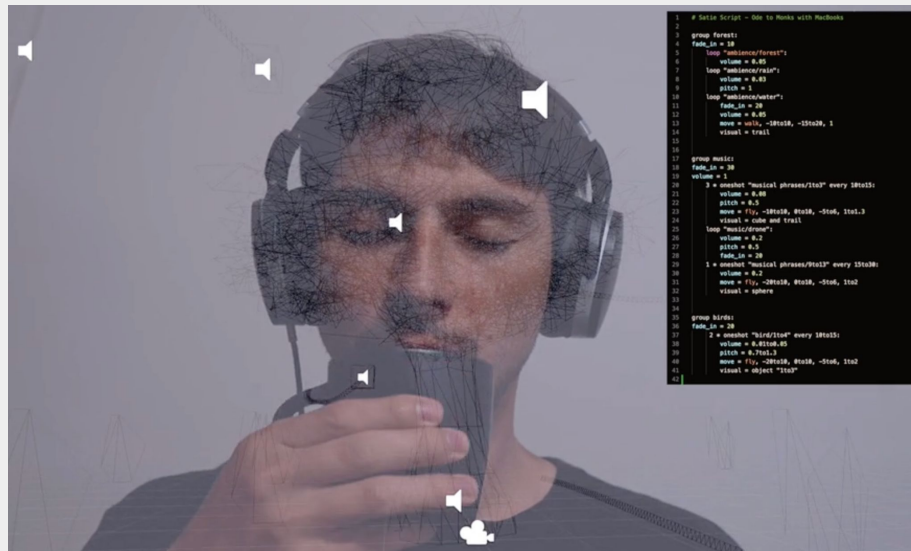
[WATCH: forest soundscape - through language](#)

How to use Satie? → through natural language

sAtle is an AI coding assistant for **Satie**. It works directly with the DSL and the audio-generation module to turn plain instructions into runnable sound behaviors.

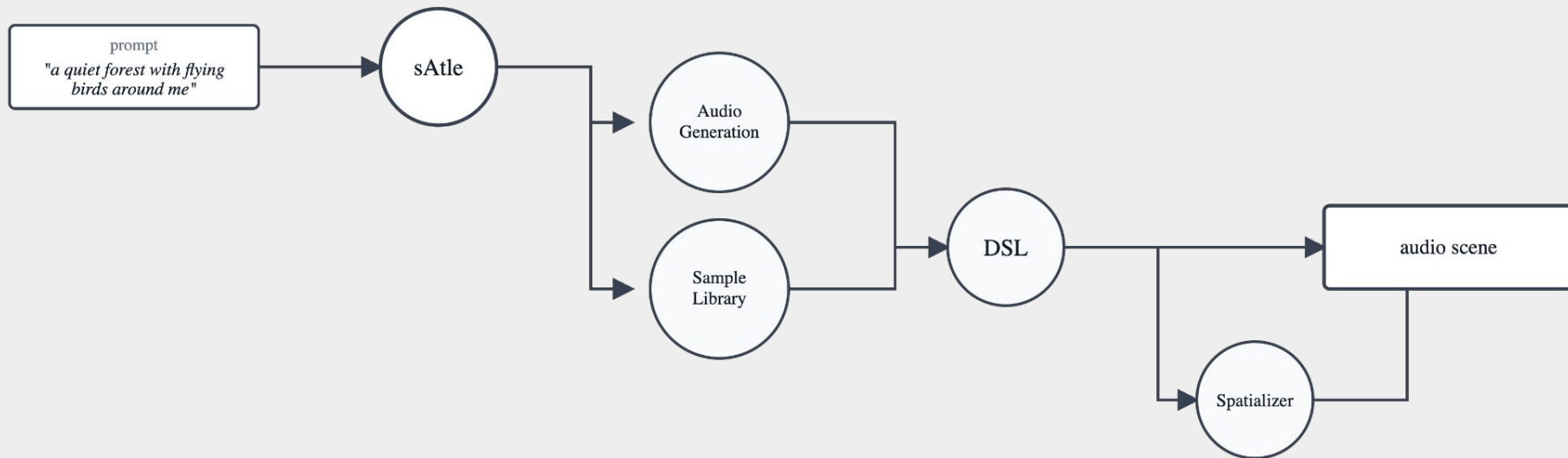
What it does

1. **Orchestrates behavior & logic:** Writes and edits Satie code, builds signal/scene logic, and validates timing/spatial rules.
2. **Completes your sample set:** Scans the user's library, maps assets to script needs, and **generates missing samples** to satisfy the request (prompt).
3. **Manages versions:** Tracks script history with checkpoints so the user can **revert** or compare changes at any time.

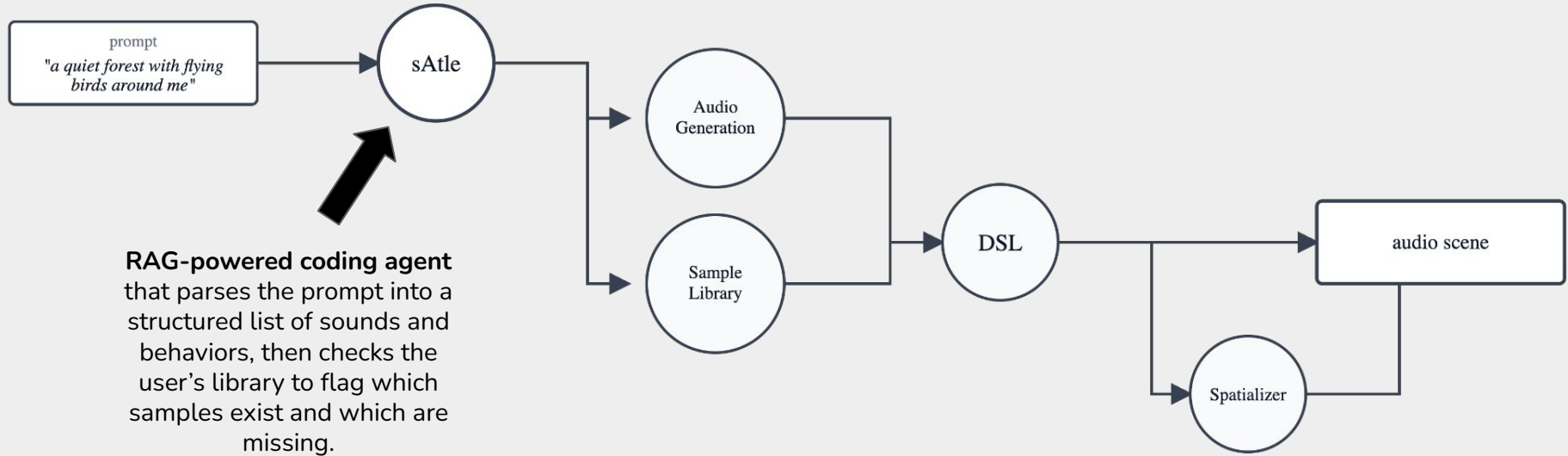


[WATCH: forest soundscape - through sAtle \(AI Agent\)](#)

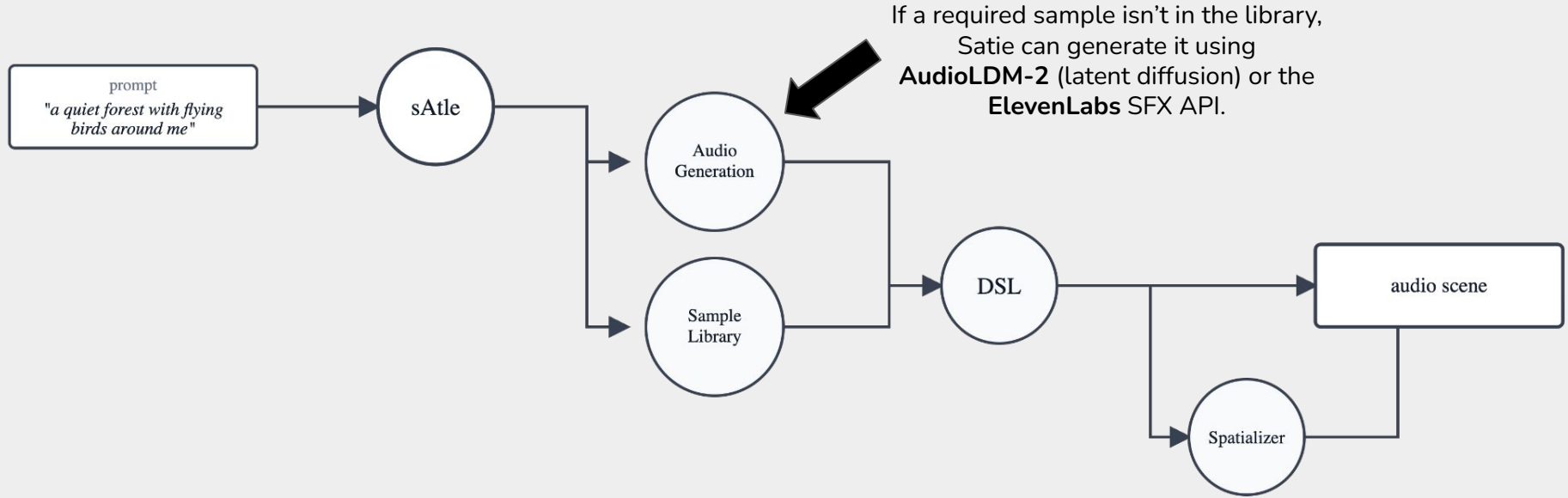
system overview



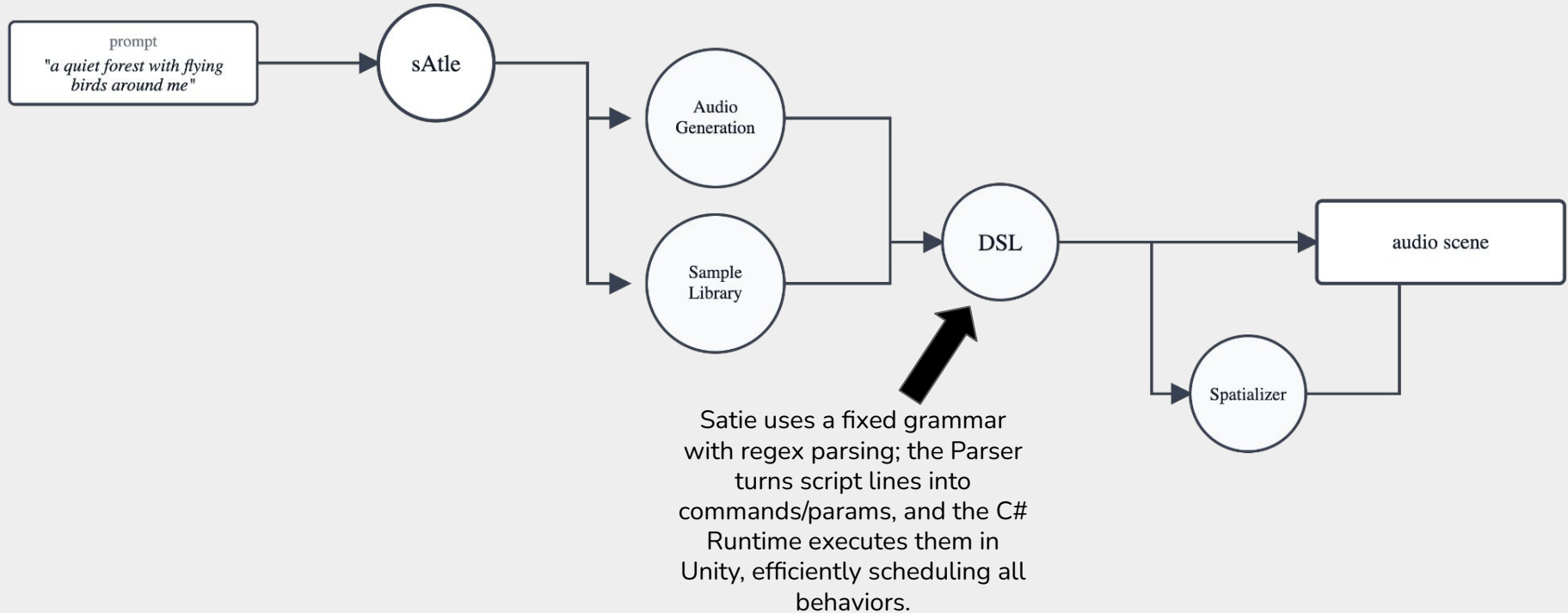
system overview



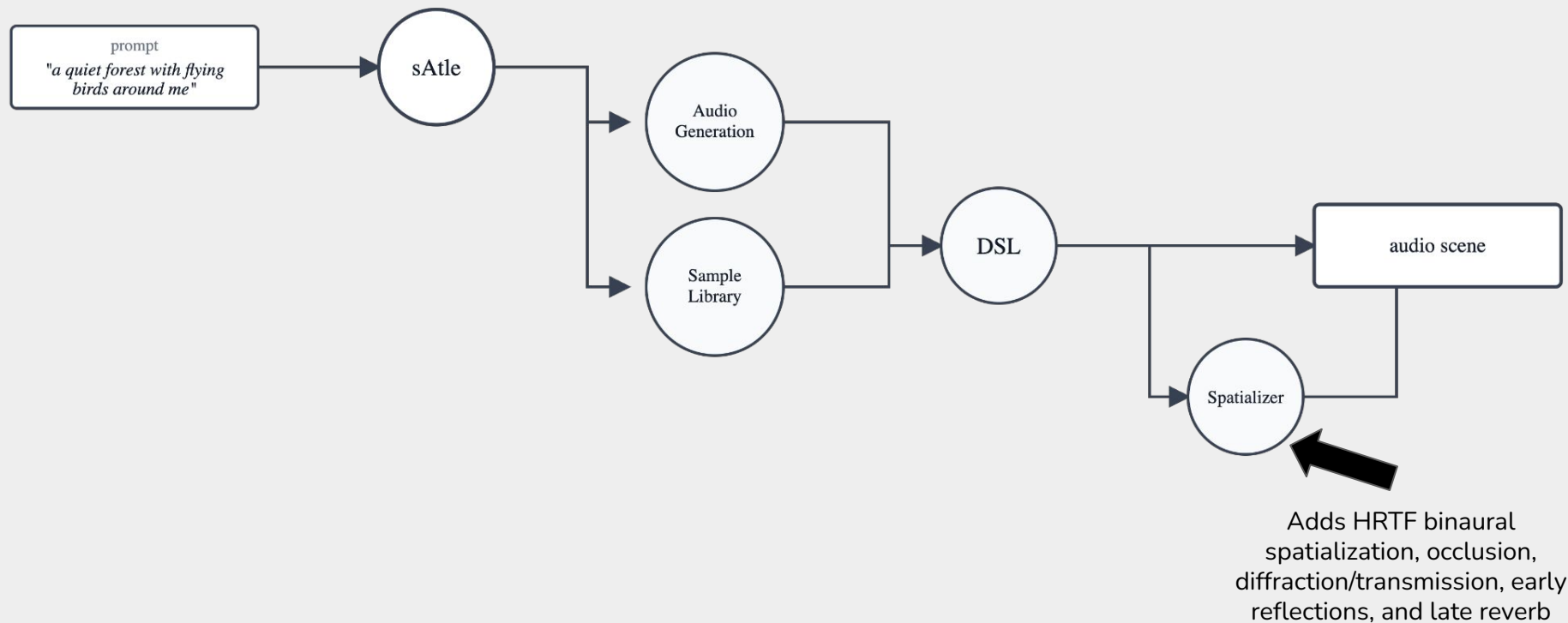
system overview



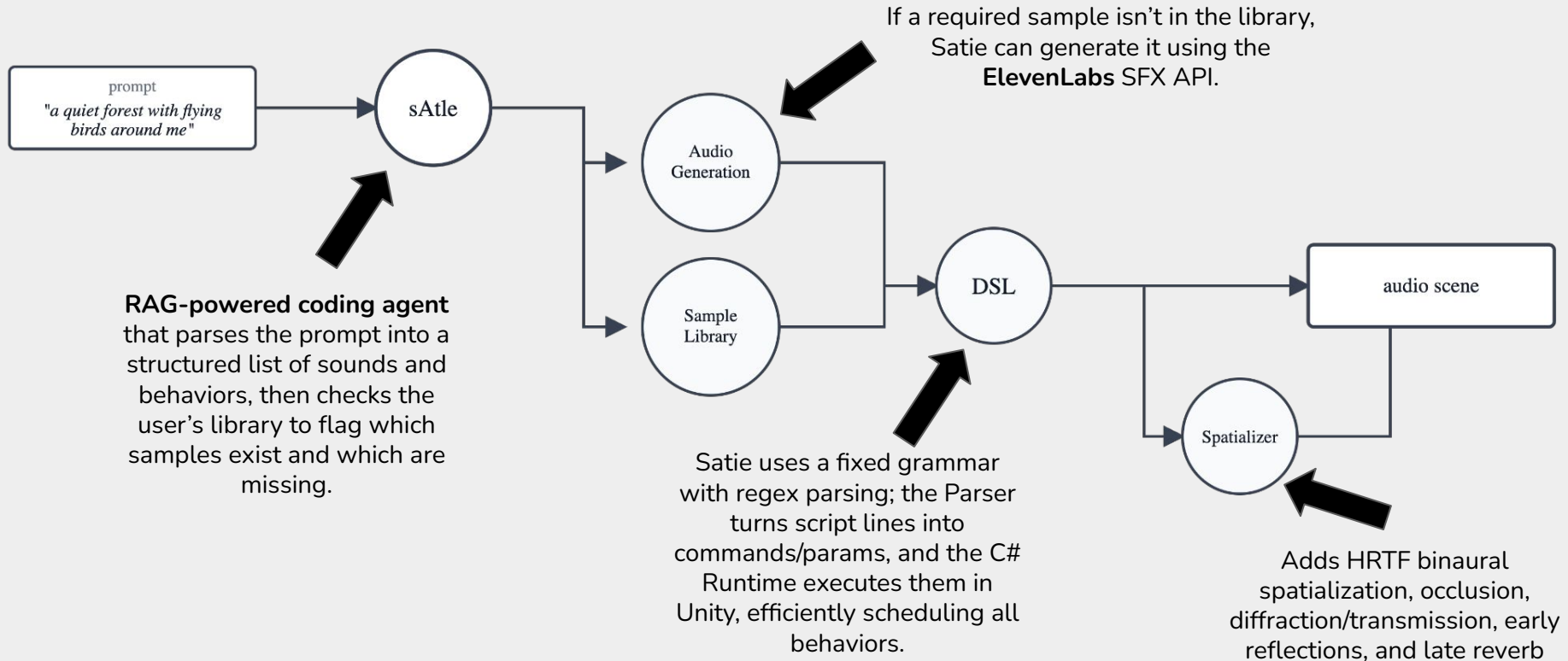
system overview



system overview



system overview



future of satie → technical level

- **Deterministic timing core:** sample-accurate scheduler (DSP timebase), temporal graph, and seeded randomness for reproducible renders.
- **Spatial audio first-class:** out-of-the-box Steam Audio routing + simple DSL verbs for occlusion, reflections, and late reverb; one-line room definitions.
- **Shipping paths:** Unity package + headless CLI; export stems, JSON manifests (provenance), and baked spatial IRs.
- **WASM sandbox:** run small scenes in the browser for reviews and playtesting.

future of satie → research

- Evaluate whether Satie improves **speed**, **control**, and **creative outcomes** for spatial sound scene building vs common workflows.